

SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

Experiments

Code of Conduct:

Shaik Hafeeza Tarannum
I 192372044 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING
ASSESSMENT TOOLS

- 1 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
 $S \rightarrow 0A1 \quad A \rightarrow 0A \mid 1A \mid \varepsilon$
- 2 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
 $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$
- 3 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
 $S \rightarrow 0S0 \mid A \quad A \rightarrow 1A \mid \varepsilon$
- 4 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
 $S \rightarrow 0S1 \mid \varepsilon$
- 5 Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)
 $S \rightarrow A101A \quad A \rightarrow 0A \mid 1A \mid \varepsilon$

Aim: To write a c program to check whether a given string belongs to the language defined by CFG.

$S \rightarrow 0A1$

$A \rightarrow 0A1 \mid A1 \mid \epsilon$

procedure:

1. Start the program
2. Read the input string
3. Analyze the string according to given CFG production rules.
4. Check whether the string's can be derived for 'S' start
5. If string can be derived, accept it (or) Reject it
6. Display the result
7. Stop the program.

code:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    printf("Enter a string:");
```

```
    scanf("%s", str);
```

```
    int len = strlen(str);
```

```
    if (len >= 2 && str[0] == '0' && str[len-1] == '1') {
```

```
        printf("String belongs to the language.\n");
```

```
    } else {
```

```
        printf("String does not belong to the language.\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Result:

Enter a string: 01

String belongs to the language

$\therefore S \rightarrow 0A1, A \rightarrow 0A1 \mid A1 \mid \epsilon$ are accepted

2A) Aim: To write a C program to check whether a given string belong to CFG.

$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

Procedure:

1. Start the program
2. Read the input string
3. Compare the first and last symbols of the string
4. Recursively check the remaining substring
5. If the symbols match until the string becomes empty, accept it
6. Otherwise reject the string
7. Stop the program.

Code:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int ispalindrome (char str[], int start, int end) {
```

```
    if (start >= end)
```

```
        return 1;
```

```
    if (str[start] != str[end])
```

```
        return 0;
```

```
    return ispalindrome (str, start+1, end-1);
```

```
}
```

```
int main() {
```

```
    char str[100];
```

```
    printf("Enter a string");
```

```
    scanf ("%s", str);
```

```
    int len = strlen(str);
```

```
    if (is palindrome (str, 0, len - 1))
```

```
        printf("String belongs to the language.\n");
```

```
    else
```

```
        printf("String does not belong to language.\n");
```

```
    return 0;
```

```
}
```

Result: Enter a string: $\epsilon, 1, 0$

string belong to language. $\therefore S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

2. string
Q. To write a C program to check whether a given string belongs to the language defined by CFG,

$S \rightarrow OSOA$

$A \rightarrow 1A1 \mid \epsilon$

$\epsilon, 1, 11, 1111, \dots$

Then $S \rightarrow OSOA$ adds the same number of 0s on both sides.

$$L = 0^n 1^m 0^n$$

Code:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int len, i, j, zerosLeft = 0, zerosRight = 0;
```

```
    printf("Enter a string");
```

```
    scanf("%s", str);
```

```
    len = strlen(str);
```

```
    i = 0;
```

```
    while (i < len && str[i] == '0') {
```

```
        zerosLeft++;
```

```
        i++;
```

```
    } while (i < len && str[i] == '1') {
```

```
        i++;
```

```
    }
```

```
    j = 1;
```

```
    while (j < len && str[j] == '0') {
```

```
        zerosRight++;
```

```
        j++;
```

```
    }
```

```
    if (i == j && zerosLeft == zerosRight) {
```

```
        printf("string belongs to the language.\n");
```

```
    } else {
```

```
        printf("string does not belong to the language.\n");
```

```
    }
```


Result: Enter a string: 01110.
String belongs to the language
∴ Accepted

(Q2) Aim: To write a C program to check whether a given string belongs to language defined by CFG.

$S \rightarrow 0S1 \mid \epsilon$

$L = 0^n 1^n = \{ \epsilon, 01, 0011, 000111 \}$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int len, zeros = 0, ones = 0;
    printf("Enter a string:");
    scanf("%s", str);
    len = strlen(str);
    i = 0;
    while (i < len && str[i] == '0') {
        zeros++;
        i++;
    }
    while (i < len && str[i] == '1') {
        ones++;
        i++;
    }
    if (i == len && zeros == ones) {
        printf("string belongs to the language.\n");
    }
    else {
        printf("string does not belong to the language.\n");
    }
    return 0;
}
```

Result: Enter a string: 000111.
String belongs to the language and is accepted

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100

SA